# Cloud Incident Monitoring and Response Platform

Mahendra Shabade | B9IS109

## Table of Contents

## 1. Introduction

This report documents a full-stack Cloud Incident Monitoring and Response Platform built for B9IS109. The stack includes a JavaScript frontend, Node.js and Express backend, MongoDB Atlas, JWT authentication, Socket.IO, and AWS EC2 deployment.

## 2. Project Overview

Cloud teams need one register for incidents instead of scattered alerts. The project delivers login, dashboard, CRUD, search, webhooks, RBAC, and responsive UI as a complete web information system.

## 3. System Requirements

Functional: secure login, incident CRUD, search and filters, dashboard, webhook ingestion, timeline audit trail. Non-functional: Node.js, MongoDB Atlas, HTTP status codes, JWT and RBAC, responsive UI, cloud deployment, central error handling.

## 4. System Architecture

Client-server design (Fielding, 2000). Express exposes /api/auth, /api/incidents, /api/observability, /api/cost. Frontend uses Fetch API and Socket.IO. Production: Nginx, PM2, EC2, MongoDB Atlas.

## 5. Data Model

Incidents store incidentId, severity, status, provider, service, region, SLA fields,
timeline, and comments. Users store hashed passwords and roles Guest, User, Administrator.

## 6. CRUD Operations

POST create returns 201. GET list and GET by id. PUT update and POST comments. DELETE admin
only. Socket.IO broadcasts changes.

## 7. Additional Features

Search, pagination, Chart.js dashboard, SLA timer, badges, JWT/RBAC, responsive CSS, theme toggle, real-time sync.

## 8. Testing and Validation

Manual tests for login, CRUD, RBAC, webhook, health endpoint, and Socket.IO. Automated Jest tests planned as future work.

## 9. External API Integration

Optional COST_API_URL for external billing JSON. Inbound webhook at /api/incidents/webhook with shared secret for cloud alerts.

## 10. Conclusion

The platform meets B9IS109 requirements with full incident CRUD, dashboards, security, and AWS deployment as a complete full-stack deliverable.

## Reflective Learning Experience

The project connected frontend, API, and database into one system. EC2 deployment and Socket.IO were the main learning areas. I would add automated tests earlier next time.